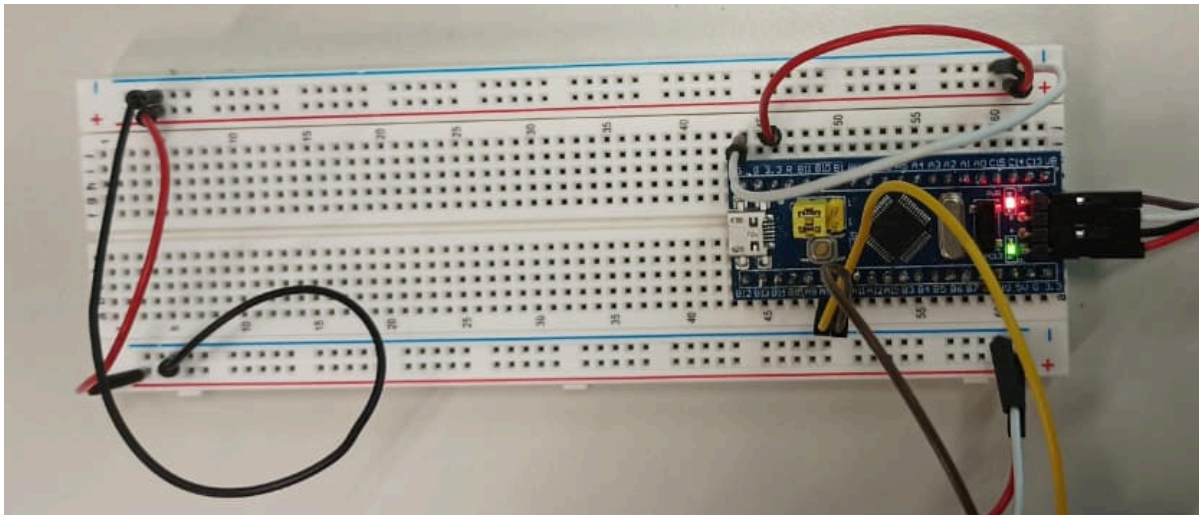


Protocolo Serie

Con framing y checksum sobre UART en STM32F103

Framing y Checksum.....	2
Cálculo de la longitud.....	3
Cálculo del checksum.....	3
Validación del protocolo.....	4
Parser incremental.....	5
¿Por qué es incremental?.....	6
Retorno de carro.....	6
Tramas pegadas.....	6
Estado, errores y resincronización.....	8
Contadores internos.....	9
Resincronización.....	9
Trama status.....	10
Conclusiones técnicas.....	11
Checksum XOR vs. otros mecanismos:.....	11
Longitud y temporización:.....	11
Resincronización completa:.....	11
Conexionado y niveles de tensión:.....	12



- Alumno: Juan Ignacio Borda Rojas
- Docente: Leonardo Javier Amet
- Fecha de entrega: 08/07/2026

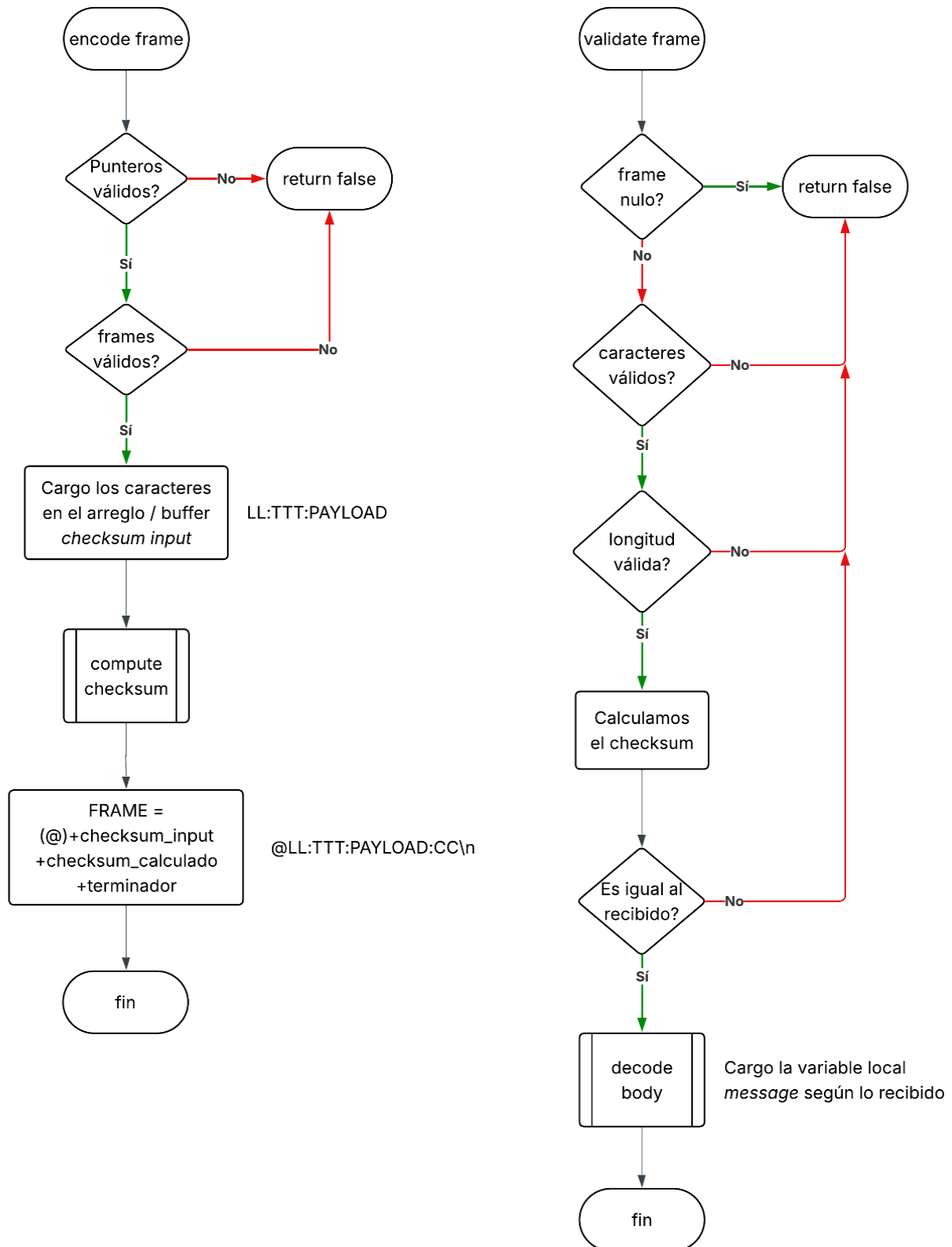
Repositorio:

https://github.com/jiborjas/TP5_Borda_Rojas.git

https://github.com/jiborjas/TP5_Borda_Rojas/tree/main/diagramas

https://github.com/jiborjas/TP5_Borda_Rojas/tree/main/tests/host

Framing y Checksum



https://github.com/jiborjas/TP5_Borda_Rojas/blob/main/firmware/protocol/protocol.c

Cálculo de la longitud

La elección de que LL cuente estrictamente el cuerpo (TTT:PAYLOAD) permite independizar la longitud de los datos de su "envoltura" o framing. Esta estructura simplifica el parseo incremental al definir exactamente cuántos bytes debe acumular el receptor antes de procesar el checksum, evitando tener que recalcular longitudes si se decidiera cambiar el carácter de inicio @ o agregar nuevos separadores.

Si bien incluir los campos externos (@, :, CC, \n) en la cuenta facilita el uso de transferencias por DMA o buffers de tamaño fijo al conocer la dimensión total del paquete de antemano, esto obligaría al parser a conocer el largo de los campos que justamente sirven para delimitar la trama. En consecuencia, integrarlos aportaría información redundante que ya está definida por la función de cada delimitador, complicando la lógica interna del sistema sin un beneficio operativo claro.

Cálculo del checksum

Si se incluyera el carácter de inicio @ en el cálculo del checksum, el valor resultante sería necesariamente distinto debido a la sensibilidad de la operación XOR ante cualquier bit adicional en el acumulado. Este cambio impediría el funcionamiento con el bridge provisto por la cátedra, ya que el mismo espera una validación de integridad basada estrictamente en los campos LL:TTT:PAYLOAD. Por ejemplo, en el comando ping, el checksum estándar calculado sobre 08:CMD:ping es 0x52; al aplicar la operación XOR adicional con el valor ASCII del @ (0x40), el nuevo código de control pasaría a ser 0x12, invalidando la trama para el receptor.

Para calcularlo, primero debemos armar el buffer que servirá a modo de input. Podemos hacerlo de la siguiente manera:

```
checksum_input[0] = nibble_to_hex((uint8_t) (body_length >> 4));
checksum_input[1] = nibble_to_hex((uint8_t) body_length);
checksum_input[2] = PROTOCOL_SEPARATOR_CHAR;

// written representa el largo de la cadena TTT:PAYLOAD.
written = snprintf(&checksum_input[3], sizeof(checksum_input) -
3U, "%s:%s", type_text, message->payload);
if ((written < 0) || ((size_t) written != body_length)) {
    return false;
} // confirmamos que el cuerpo se grabó completo en el buffer
```

Con el buffer cargado, resta calcularlo con checksum = protocol_compute_checksum(checksum_input, checksum_input_length);

```
Luego, uint8_t protocol_compute_checksum(const char *data, size_t length)
{for (i = 0U; i < length; i++) {checksum ^= (uint8_t) data[i];} return checksum;}
```

La función snprintf es variádica, lo que significa que puede recibir tantos argumentos como se especifiquen en la plantilla de formato. Al ejecutarse, el buffer checksum_input queda con la cadena completa lista para el XOR (ej. 08:CMD:ping).

De forma genérica, podemos describirla de la siguiente manera:

```
retorno = snprintf(puntero_destino, capacidad_maxima, "plantilla", dato_A, dato_B);
```

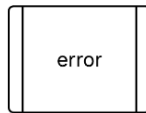
1. **Retorno:** Es un int que guarda cuántos caracteres intentó escribir la función (sin contar el \0 final). Si este número es igual a la longitud que se espera (body_length), significa que el mensaje entró completo y no se cortó por falta de espacio.
2. **puntero_destino (char*):** La dirección de memoria del buffer donde se empezará a guardar el texto
3. **capacidad_maxima (size_t):** El límite de seguridad para evitar desbordamientos de memoria (buffer overflow).
4. **formato (const char*):** La "plantilla". Por ejemplo: "%s:%s" para poner dos textos separados por dos puntos).
5. **argumentos:** Son las variables reales que se insertan en el formato. Por ejemplo: el tipo (TTT) y el contenido (PAYLOAD).

Finalmente, realizamos un XOR bit a bit: **checksum ^= data[i];** hasta obtener el resultado de nuestro checksum.

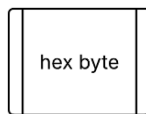
Validación del protocolo

La función `protocol_validate()` debe buscar obligatoriamente el último caracter separador : para garantizar la correcta extracción del código de control. Si se buscara el primero, la validación fallaría en situaciones donde el payload del mensaje contiene dos puntos como parte de sus propios datos. En estos casos, buscar el primer separador provocaría que el parser intente procesar el resto del cuerpo del mensaje como si fuera el checksum, resultando en un error de validación inmediato. Por el contrario, localizar el último asegura que se aíslen correctamente los dos dígitos hexadecimales finales destinados a la verificación de integridad.

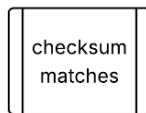
Parser incremental



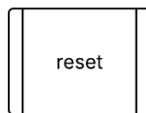
gestiona el error
y resincroniza



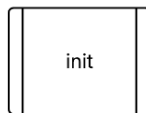
Convierte dos caracteres hexa
en un valor numérico de 8 bits



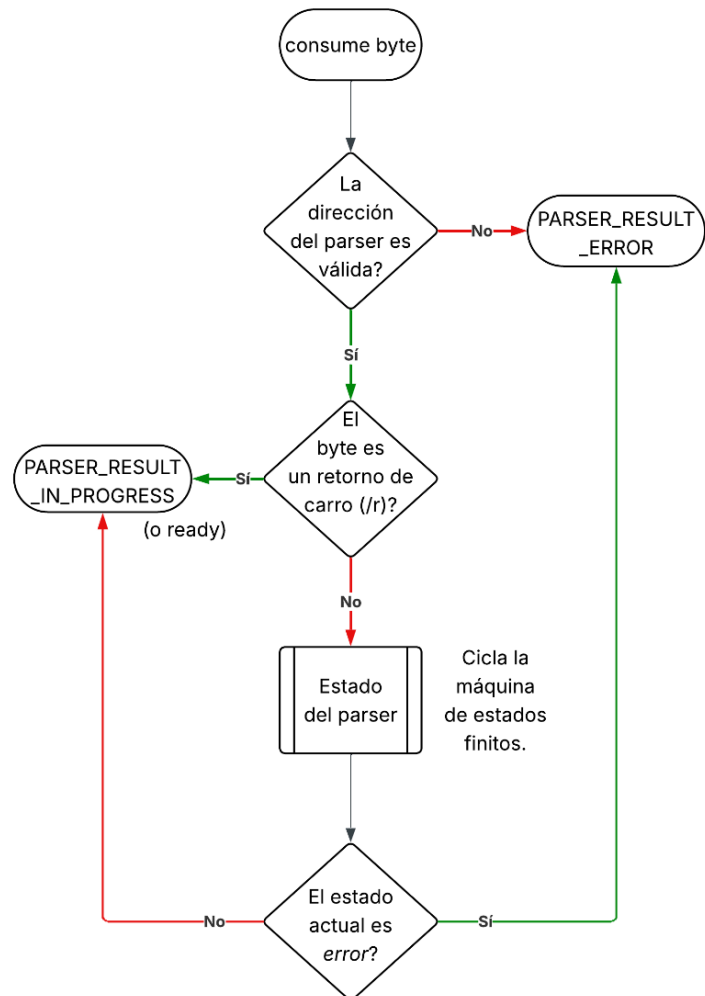
Valida que el checksum
coincida con el calculado



Carga el parser con
su estado inicial



llama a reset



https://github.com/jiborjas/TP5_Borda_Rojas/blob/main/firmware/protocol/parser.c

```

COM4 - PuTTY
@0B:CMD:status?:13
@29:STS:rx=1,ae=0,irq=19,pb=19,pm=1,pe=0,qd=0:39
@08:CMD:ping:52
@0A:ACK:pong=1:22
safdfdsagg@08:CMD:ping:52
@0A:ACK:pong=1:22
@0A:CMD:led=on:6A
@0A:ACK:cmd=ok:6B
@0B:CMD:led=off:07
@0A:ACK:cmd=ok:6B
@0E:CMD:led=toggle:7D
@0A:ACK:cmd=ok:6B
@XX:CMD:loquesea:YY@0C:CMD:loquesea:2C
@14:ERR:code=unknown_cmd:2D
@0B:CMD:status?:13
@2B:STS:rx=8,ae=1,irq=178,pb=178,pm=8,pe=2,qd=0:41
  
```

¿Por qué es incremental?

El parser debe ser incremental (byte a byte) porque la UART entrega un flujo continuo de datos y no garantiza la preservación de "mensajes" completos. Es decir, microcontrolador no sabe automáticamente dónde empieza o termina una trama dentro de ese flujo. Una lectura por línea con un buffer fallaría en las siguientes situaciones concretas:

1. **Ruido o basura antes del inicio:** Si llegan bytes aleatorios antes del carácter @, un lector de líneas podría intentar procesar toda la cadena como un solo bloque, rechazándola por formato inválido en lugar de ignorar la basura y buscar el comienzo real de la trama.
2. **Tramas fragmentadas:** Si una trama llega dividida en varias interrupciones o lecturas parciales, el sistema podría quedar bloqueado esperando un terminador que tarda en llegar o procesar un buffer incompleto, mientras que el parser incremental puede acumular los datos a medida que llegan sin detener la ejecución.
3. **Tramas pegadas:** Si se reciben dos tramas consecutivas sin separación (ej. @08...:52\n@0A...:6A\n), un lector de líneas podría tratarlas como una única cadena larga y fallar en la validación de longitud y checksum.

Retorno de carro

Si no se ignorara el carácter \r (0x0D), el error ocurriría principalmente en el estado EXPECT_END.

En este estado, el parser ya ha procesado el checksum y espera recibir exactamente un \n (0x0A) para cerrar y validar la trama. Si llega un \r antes del \n (algo muy común en terminales que envían la secuencia \r\n), el parser lo interpretaría como un carácter inválido, descartaría la trama completa por error de formato y volvería a WAIT_START.

Tratarlo como un byte normal también causaría fallas en estados que esperan únicamente dígitos hexadecimales o el separador, ya que \r no cumple con ninguno de esos roles.

Tramas pegadas

Como se mencionó anteriormente, sí, el parser entrega la segunda trama correctamente y sin pérdida de datos.

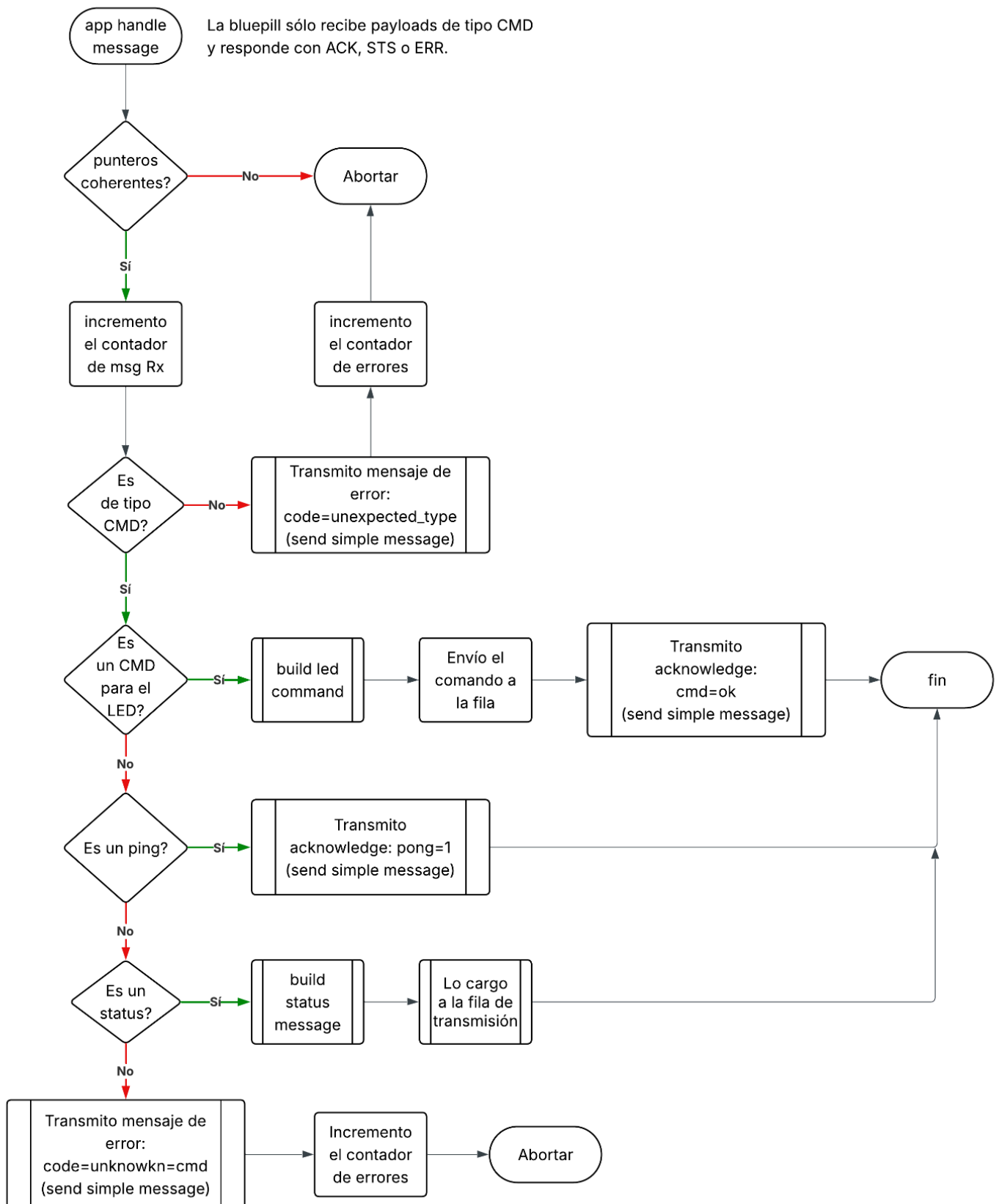
Esto funciona porque el parser es reactivo al formato (@ y \n) y no depende de pausas de tiempo entre mensajes. Al recibir el terminador \n de la primera trama, el parser valida el mensaje, lo entrega a la aplicación y vuelve automáticamente al estado WAIT_START. Como el siguiente byte en el flujo de la UART es el @ de la nueva trama, el proceso se reinicia de inmediato.

Trazado de estados para la segunda trama (@0A:CMD:led=on:6A\n):

1. @: WAIT_START → READ_LEN_HI (detecta inicio).
2. 0: READ_LEN_HI → READ_LEN_LO (guarda LL).
3. A: READ_LEN_LO → EXPECT_LEN_SEPARATOR (guarda LL, longitud = 10).
4. ': EXPECT_LEN_SEPARATOR → READ_BODY (separador OK).
5. CMD:led=on: READ_BODY (acumula los bytes) → EXPECT_CHECK_SEPARATOR.
6. ': EXPECT_CHECK_SEPARATOR → READ_CHECK_HI (separador OK).

7. 6: READ_CHECK_HI → READ_CHECK_LO (guarda CC).
8. A: READ_CHECK_LO → EXPECT_END (guarda CC).
9. \n: EXPECT_END → MESSAGE_READY (valida checksum y entrega mensaje).

Estado, errores y resincronización



https://github.com/jiborjas/TP5_Borda_Rojas/blob/main/firmware/app/app.c

Contadores internos

Es fundamental reportar el estado mediante contadores internos en lugar de depender únicamente de los registros de la USART, ya que el hardware solo ofrece una visión instantánea del sistema (como la presencia de un byte o un error de overrun). En contraste, los contadores describen el comportamiento acumulado del software a lo largo del tiempo, permitiendo un diagnóstico preciso por capas.

Esta estrategia facilita identificar en qué nivel ocurre una falla: por ejemplo, si el contador de interrupciones (irq) aumenta pero el de mensajes válidos (pm) no, se deduce que la UART recibe datos físicos pero el parser los está rechazando por errores de framing o checksum. Además, los contadores proveen información lógica que el hardware no puede reportar, como errores por comandos desconocidos (ae) o bytes descartados específicamente por colas de FreeRTOS llenas (qd), lo cual diferencia claramente un problema de gestión de memoria de una falla física en el chip.

Resincronización

Si el parser NO reutilizara el @ del error, entregaría 0 mensajes. Al no aprovechar ese caracter como un nuevo inicio, descarta la única marca de comienzo válida para la trama real, quedando "sordo" ante el resto de los bytes.

Trazado de estados sin reutilización:

1. Byte @: WAIT_START → READ_LEN_HI (Detecta inicio de trama inválida).
2. Byte 0: READ_LEN_HI → READ_LEN_LO (Guarda primer dígito de longitud).
3. Byte @: Error en READ_LEN_LO (se esperaba un dígito hexadecimal). El parser vuelve a WAIT_START y descarta el byte @ actual.
4. Bytes 0, 8, :, C, M, D...: El parser permanece en WAIT_START ignorando todos estos bytes, ya que ninguno es un @.
5. Byte \n: Se ignora en WAIT_START. El sistema termina sin haber validado ninguna trama.

```

COM4 - PuTTY
@0B:CMD:status?:13
@29:STS:rx=1,ae=0,irq=19,pb=19,pm=1,pe=0,qd=0:39
@08:CMD:ping:52
@0A:ACK:pong=1:22
safdfdsagg@08:CMD:ping:52
@0A:ACK:pong=1:22
@0A:CMD:led=on:6A
@0A:ACK:cmd=ok:6B
@0B:CMD:led=off:07
@0A:ACK:cmd=ok:6B
@0E:CMD:led=toggle:7D
@0A:ACK:cmd=ok:6B
@XX:CMD:loquesea:YY@0C:CMD:loquesea:2C
@14:ERR:code=unknown_cmd:2D
@0B:CMD:status?:13
@2B:STS:rx=8,ae=1,irq=178,pb=178,pm=8,pe=2,qd=0:41

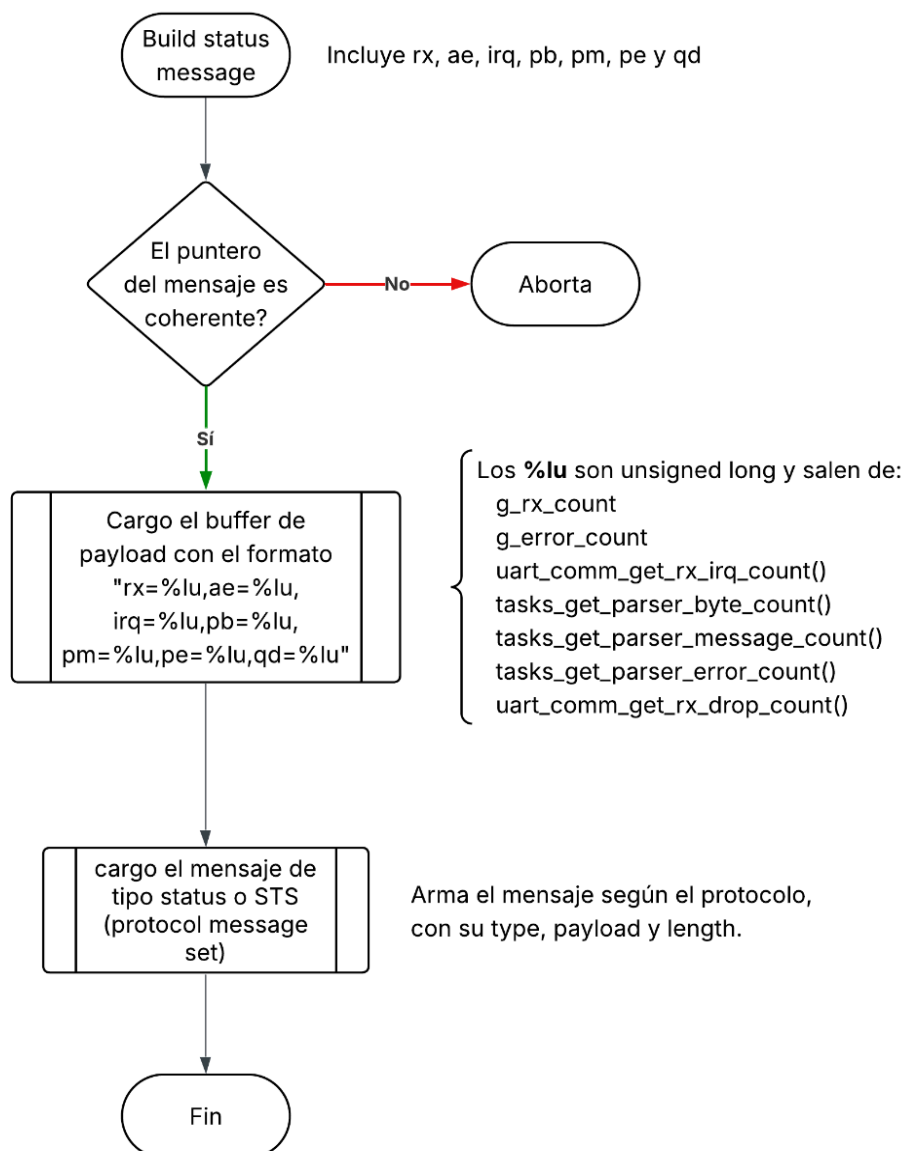
```

Trama status

Al recibir una trama de tipo STS, el bridge publica la información en el tópico bridge/status. El contenido publicado será el texto del payload recibido, como por ejemplo rx=5,ae=2,...

Se utiliza el tipo STS (Status) y no ACK porque la respuesta no es una simple confirmación de que el comando se recibió, sino que transporta un reporte con los siguientes contadores de diagnóstico internos del sistema:

1. rx = mensajes recibidos por la aplicación
2. ae = errores de aplicación
3. irq = bytes recibidos por interrupción UART
4. pb = bytes procesados por el parser
5. pm = mensajes válidos entregados por el parser
6. pe = errores detectados por el parser
7. qd = bytes descartados por cola llena



Conclusiones técnicas

Checksum XOR vs. otros mecanismos:

El protocolo implementado utiliza una operación XOR byte a byte como mecanismo de verificación de integridad. Este método es eficaz para detectar errores simples, como la alteración de un solo bit o un solo byte completo dentro de la trama, aunque presenta limitaciones técnicas importantes. Una de las principales debilidades es que el checksum XOR no detecta el intercambio de posición entre dos bytes del cuerpo, ya que la operación es conmutativa ($A \oplus B = B \oplus A$) y produce un acumulado final idéntico sin importar el orden de los caracteres. Del mismo modo, el sistema no detecta errores múltiples si las alteraciones en diferentes bytes ocurren en las mismas posiciones de bits, provocando que los cambios se cancelen mutuamente en el resultado final.

Para obtener una mayor robustez sin alterar la estructura textual de la trama (@LL:TTT:PAYLOAD:CC\n), la alternativa ideal es el CRC-8 (verificación de redundancia cíclica). A diferencia del XOR, el cálculo de CRC depende estrictamente del orden de los datos, lo que permite detectar inmediatamente el intercambio de bytes. Dado que el resultado de un CRC-8 sigue siendo de 8 bits, se puede representar con los mismos dos caracteres hexadecimales en el campo CC, manteniendo la compatibilidad visual y de tamaño, aprovechando además que el STM32F103 incluye una unidad de hardware dedicada para acelerar estos cálculos de integridad.

Longitud y temporización:

Considerando una configuración de 115200 baudios con formato 8N1, cada carácter transmitido requiere un total de 10 bits, compuestos por 1 bit de inicio, 8 de datos y 1 de parada. Para una trama de longitud máxima de 64 caracteres, esto representa una carga total de 640 bits, resultando en un tiempo de transmisión de aproximadamente 5,56 milisegundos.

En el caso de recibir una trama donde el campo de longitud indique un valor de FF (255 decimal) pero el cuerpo real sea menor, el firmware implementado rechaza la operación inmediatamente al detectar que el valor excede el límite definido por PROTOCOL_MAX_BODY_SIZE, reiniciando el parser para esperar una nueva trama válida. Si el sistema no validara este límite superior, el parser permanecería en estado de lectura esperando completar los 255 bytes declarados, lo que provocaría una desincronización y la pérdida de mensajes subsiguientes hasta que ocurra un error de formato o un reinicio por ruido. Este escenario no debe presentarse en una operación normal, ya que el protocolo exige que el emisor calcule con precisión el campo LL basándose estrictamente en el cuerpo TTT:PAYLOAD y respetando siempre el tamaño máximo permitido para garantizar la fluidez de la comunicación.

Resincronización completa:

Al procesar la secuencia ###@08:CMD:ping:52\n, el parser se encuentra inicialmente en el estado WAIT_START, donde ignora consecutivamente los tres caracteres # por no representar marcas de inicio de trama válidas. Al recibir el carácter @, la máquina

de estados realiza la transición a `READ_LEN_HI`, avanzando luego por `READ_LEN_LO` con el dígito 0 y hacia `EXPECT_LEN_SEPARATOR` con el 8 para establecer una longitud de cuerpo de 8 bytes. El parser atraviesa el estado `READ_BODY` acumulando la cadena `CMD:ping`, confirma el separador antes del checksum y procesa los dígitos hexadecimales 5 y 2 para finalmente validar la integridad en `EXPECT_END` al recibir el carácter `\n`, entregando así un único mensaje válido.

Este escenario difiere del caso `@0@08:CMD:ping:52\n` debido a que en este último la basura inicial inicia una trama parcial que falla al recibir la segunda arroba en lugar del dígito hexadecimal esperado. A diferencia de los caracteres `#` que se descartan pasivamente en el estado inicial, la segunda `@` en medio de una trama rota dispara un error de parser que activa la lógica de resincronización, permitiendo reutilizar dicho carácter como un nuevo inicio de trama para recuperar con éxito el mensaje completo que le sigue.

Conexionado y niveles de tensión:

No resulta seguro conectar directamente el pin PA10 a la salida TX de un adaptador USB-UART que trabaje con lógica de 5 V, debido a que el STM32F103 opera internamente a 3,3 V y este exceso de tensión puede dañar o estresar permanentemente el pin del microcontrolador. Para garantizar la integridad del hardware, es imperativo utilizar un adaptador TTL configurado a 3,3 V o, en su defecto, interponer un convertor de niveles o divisores resistivos que adecuen las señales a los límites eléctricos del dispositivo.

Es importante destacar que esta limitación es estrictamente de naturaleza física y no tiene impacto alguno sobre la lógica del protocolo, por lo que el formato de las tramas y el cálculo de los checksums permanecen inalterados siempre que los niveles de tensión permitan una correcta interpretación de los bits por parte de la UART.